

Analysis of Stress Distribution in a Cantilever Beam

Group 12

Kush Mehta (24110177), Khushal Manani (24110191),
Shubham Chandak (24110335), Sunay Desai (24110361)

I. OBJECTIVES

The objective of this project is to analyze the stress distribution in a cantilever beam under different loading conditions. The specific objectives are:

- To compute the shear force and bending moment along the beam.
- To calculate normal and shear stresses at any point in the cross-section.
- To study stress transformation at different orientations.
- To represent the stress state using Mohr's Circle.

II. PROBLEM STATEMENT

A cantilever beam of circular cross-section is fixed at one end and subjected to point loads and uniformly distributed loads along its length. The analysis involves determining the variation of shear force and bending moment at different sections of the beam.

The bending stress at any point in the cross-section is given by $\sigma = \frac{My}{I}$, where M is the bending moment, y is the distance from the neutral axis, and I is the Second moment of area. The shear stress is given by $\tau = \frac{VQ}{Ib}$, where V is the shear force, Q is the first moment of area, and b is the width at the point of interest.

The state of stress at a point is further examined using stress transformation equations. Mohr's Circle is used to represent the stress state graphically and to determine principal stresses and maximum shear stress.

III. BEAM AND STRESS DESCRIPTION

- **Fixed Point:** The beam is fixed at the left end, which is taken as the origin $(0, 0)$. We measure $+x$ along the beam's length and $+y$ vertically upwards.
- **Bending Stress (σ_x):** The bending moment stretches or compresses fibers along the beam's length, resulting in normal stress in the x -direction.
- **Shear Stress (τ_{xy}):** Shear forces try to slide vertical sections of the beam relative to each other, hence shear stress is produced due to them.
- **Simplification:** Only the forces planar in the xy -direction are considered, so there are no forces creating a tendency for stretching or compression in the y -direction. Hence $\sigma_y = 0$. There is also no twisting or side-loading considered, so all other stress components are zero in a similar manner.
- **Material Assumption:** The beam is considered weightless, linearly elastic, isotropic and slender ($d/L \leq 0.1$).
- **Sign Convention:** We define positive shear and bending moments as the internal forces and moments exerted on the left section of the beam by the right section.

- **Cross-Sectional Properties:** For the circular cross-section of radius r , the area moment of inertia about the neutral axis is given by:

$$I = \frac{\pi r^4}{4}$$

The first moment of area Q at a vertical distance y from the neutral axis is given by integrating $y'dA$ from y to r . Since $dA = 2\sqrt{r^2 - (y')^2} dy'$, the integration is:

$$Q = \int_y^r y' dA = \int_y^r y' \left(2\sqrt{r^2 - (y')^2} \right) dy'$$

Evaluating this integral finally gives Q which is used in the shear stress calculation:

$$Q = \left[-\frac{2}{3}(r^2 - (y')^2)^{3/2} \right]_y^r = \frac{2}{3}(r^2 - y^2)^{3/2}$$

IV. CODE ANALYSIS

A. Input for Users

```

1 L = float(input("Beam length (mm): "))
2 D = float(input("Beam diameter (mm): "))
3
4 L, D = L/1000, D/1000 # converting mm to m
5
6 if (D/L) > 0.1:
7     print(f"WARNING: d/L = {D/L:.4f} > 0.1,
8         not slender!")
9
10 r = D/2
11 I = (np.pi * r**4) / 4
12
13 loads = [] # array for storing all forces
14 while True:
15     ans = input("Add a force? (y/n): ")
16     if ans.lower() != 'y':
17         break
18
19     type = input("Discrete (d) or continuous (c): ")
20     if type.lower() == 'd':
21         F = float(input("Force(N) (+ve is upward): "))
22         x = float(input("Dist from fixed end(mm): "))
23         loads.append({'type': 'd', 'F': F, 'x': x/1000})
24     else:
25         w = float(input("Load intensity (N/m): "))
26         x1 = float(input("Start dist from left (mm): "))
27         x2 = float(input("End dist from left (mm): "))
28         loads.append({'type': 'c', 'w': w, 'x1': x1/1000, 'x2': x2/1000})
29
30 if not loads:
31     print("No loads applied.")
32     exit()

```

Explanation:

This code takes the length and diameter of the beam as input from the user. It keeps on taking input from the user as long as the user does not enter 'n'. For each load entered, it records the position from the left end (for discrete) or starting and ending positions (for continuous).

It converts all the length quantities from mm to meters and then calculates the second moment of area for a circular cross-section. It includes a check for the slenderness ratio D/L to verify if Euler-Bernoulli beam theory is applicable.

Example Input/Output Parameters:

```

Beam length (mm): 1000
Beam diameter (mm): 50
Add a force? (y/n): y
Discrete (d) or continuous (c): d
Force(N): 500
Distance from left end (mm): 250
Add a force? (y/n): y
Discrete (d) or continuous (c): c
Load intensity (N/m): -500
Start distance (mm): 0
End distance (mm): 500
Add a force? (y/n): n

```

B. SFD and BMD Calculation

```

1 def sfd_bmd(x):
2     shear = 0
3     bending_moment = 0
4
5     for load in loads:
6         if load["type"] == "d":
7             if load["x"] > x:
8                 shear += load["F"]
9                 dist = load["x"] - x
10                bending_moment += load["F"] *
11                dist
12            else:
13                st = max(load["x1"], x)
14                end = load["x2"]
15                if end > st:
16                    length = end - st
17                    resultant = load["w"] * length
18                    shear += resultant
19                    mid = (st + end) / 2
20                    bending_moment += resultant *
21                    (mid - x)
22
23     return shear, bending_moment

```

Explanation:

This function calculates the internal shear force and bending moment at any given point x along the beam. It iterates through the list of stored loads and sums the forces (gives the shear force) and moments (gives the bending moment) on the right side.

For discrete point loads, it checks if the load position is to the right of point x and adds its contribution to the total shear and moment. For continuous distributed loads, it identifies the portion of the load acting beyond point x , converts it into an equivalent resultant force, and calculates its moment based on the distance from its equivalent force point to x .

C. Stress Calculation

```

1 def stress(x, y):
2     shear, bending_moment = sfd_bmd(x)
3     sigma_x = bending_moment * y / I
4     sigma_y = 0
5
6     if abs(y) < r:
7         Q = (2/3) * (r**2 - y**2)**1.5
8         width = 2 * (r**2 - y**2)**0.5
9         if width != 0:
10            tau_xy = (shear * Q) / (I * width)
11        else:
12            tau_xy = 0
13    else:
14        tau_xy = 0
15
16    return sigma_x, sigma_y, tau_xy

```

Explanation:

This function calculates the normal and shear stress components at any specific point (x, y) within the beam. It first calls the previously defined function to get the internal shear force and bending moment at position x .

It then uses the formula $\sigma_x = \frac{M \cdot y}{I}$ to find the horizontal normal stress (σ_x) based on the distance from the neutral axis. For the shear stress (τ_{xy}), it calculates the first moment of area (Q) and the local width (b) of the circular cross-section at height y , then uses $\tau_{xy} = \frac{VQ}{Ib}$ to calculate the shear stress. It returns these values, with $\sigma_y = 0$.

D. Plotting SFD and BMD

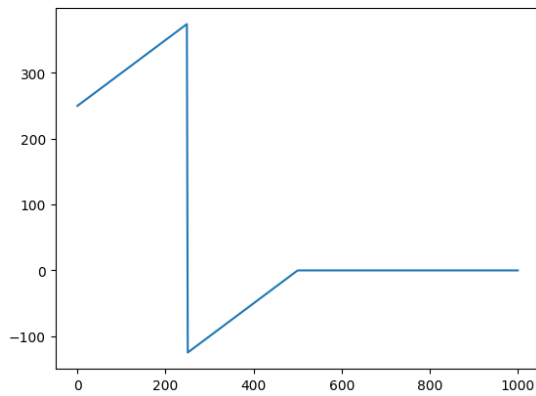
```

1 x_vals = np.linspace(0, L, 500)
2
3 sfd = []
4 bmd = []
5 for x in x_vals:
6     s, m = sfd_bmd(x)
7     sfd.append(s)
8     bmd.append(m)
9
10 plt.figure()
11 plt.plot(x_vals * 1000, sfd)
12 plt.title("Shear Force Diagram")
13
14 plt.figure()
15 plt.plot(x_vals * 1000, bmd)
16 plt.title("Bending Moment Diagram")
17
18 plt.show()

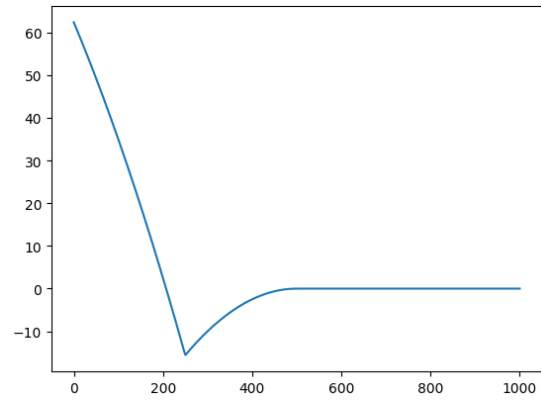
```

Explanation:

This part of the code generates the Shear Force Diagram (SFD) and Bending Moment Diagram (BMD) to visualize how internal forces change along the beam. It uses `np.linspace` to create 500 evenly spaced points across the total length L . The code then loops through each point, calling the `sfd_bmd` function to calculate the corresponding shear and moment values. Finally, it uses the Matplotlib library to plot the diagrams.



(a) Shear Force Diagram



(b) Bending Moment Diagram

Fig. 1

E. Mohr's Circle

```

1 def mohrs(sigma_x, sigma_y, tau_xy):
2     centre = (sigma_x + sigma_y) / 2
3     radius = (((sigma_x - sigma_y)/2)**2
4               + tau_xy**2)**0.5
5
6     return centre, radius

```

Explanation:

This function calculates the center and radius of Mohr's Circle defined at a specific cross-sectional point (x, y) . These values are used to draw the circle and identify the principal stresses as follows:

$$\text{Center (average normal stress): } C = \frac{\sigma_x + \sigma_y}{2}$$

$$\text{Radius (max shear stress): } R = \sqrt{\left(\frac{\sigma_x - \sigma_y}{2}\right)^2 + \tau_{xy}^2}$$

F. Rotated Stress

```

1 def rotated(sigma_x, sigma_y, tau_xy,
2             theta_deg):
3     theta = np.radians(theta_deg)
4
5     avg = (sigma_x + sigma_y) / 2
6     diff = (sigma_x - sigma_y) / 2
7
8     sigma_x_new = avg + diff*np.cos(2*theta) \
9                 + tau_xy*np.sin(2*theta)
10    sigma_y_new = avg - diff*np.cos(2*theta) \
11                - tau_xy*np.sin(2*theta)
12    tau_xy_new = -diff*np.sin(2*theta) \
13                + tau_xy*np.cos(2*theta)
14
15    return sigma_x_new, sigma_y_new,
16        tau_xy_new

```

Explanation:

This function calculates how the stresses change when we rotate our point of view by a specific angle, θ . It applies the stress transformation equations:

$$\sigma_{x'} = \frac{\sigma_x + \sigma_y}{2} + \frac{\sigma_x - \sigma_y}{2} \cos(2\theta) + \tau_{xy} \sin(2\theta)$$

$$\sigma_{y'} = \frac{\sigma_x + \sigma_y}{2} - \frac{\sigma_x - \sigma_y}{2} \cos(2\theta) - \tau_{xy} \sin(2\theta)$$

$$\tau_{x'y'} = -\frac{\sigma_x - \sigma_y}{2} \sin(2\theta) + \tau_{xy} \cos(2\theta)$$

V. RESULTS AND COMPARISON

After running the logic, three sliders are provided for x , y , and θ . The user can adjust these sliders to calculate the stress at any required configuration. The tool also provides a visual reference, showing the exact position on the beam being analyzed and the rotated state of the stress element.

To verify the code, we calculated the stress at two different configurations (x, y, θ) :

- 1) $x = 300$ mm, $y = 0$ mm, $\theta = 0^\circ$
- 2) $x = 0$ mm, $y = -15$ mm, $\theta = -30^\circ$

The outputs for these two configurations are shown below:

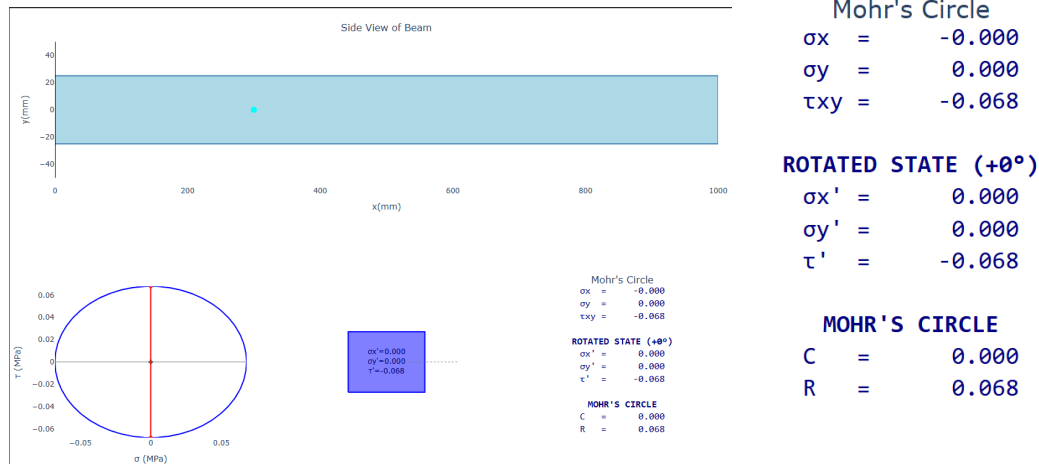


Fig. 2: Output for Case 1: (300 mm, 0 mm, 0°).



Fig. 3: Output for Case 2: (0 mm, -15 mm, -30°).

Next, to confirm these results, we calculated the shear force and bending moment analytically. Using the loading conditions, the functions are as follows:

Shear Force, $V(x)$:

$$V(x) = \begin{cases} 500 - 500(0.5 - x) & \text{for } 0 \leq x < 0.25 \text{ m} \\ -500(0.5 - x) & \text{for } 0.25 \leq x < 0.5 \text{ m} \\ 0 & \text{for } 0.5 \leq x \leq 1.0 \text{ m} \end{cases}$$

Bending Moment, $M(x)$:

$$M(x) = \begin{cases} 500(0.25 - x) - 250(0.5 - x)^2 & \text{for } 0 \leq x < 0.25 \text{ m} \\ -250(0.5 - x)^2 & \text{for } 0.25 \leq x < 0.5 \text{ m} \\ 0 & \text{for } 0.5 \leq x \leq 1.0 \text{ m} \end{cases}$$

Using these analytical functions, the normal stress (σ) and shear stress (τ) were computed manually for the two cases. As we can see in the table below, the output from the code perfectly matches the analytical output.

Configuration (x, y, θ)	Case	Base State [MPa]			Mohr's Circle [MPa]		Rotated State [MPa]		
		σ_x	σ_y	τ_{xy}	C	R	$\sigma_{x'}$	$\sigma_{y'}$	$\tau_{x'y'}$
(300 mm, 0 mm, 0°)	1	0.000	0.000	-0.068	0.000	0.068	0.000	0.000	-0.068
(0 mm, -15 mm, -30°)	2	-3.056	0.000	0.109	-1.528	1.532	-2.386	-0.670	-1.269

TABLE I: Comparison of Code Output vs Analytical Calculations.

VI. LEARNING OUTCOMES

- Learned to convert beam bending formulas and Mohr's Circle into a working Python program.
- Learned to check the code by comparing results with manual calculations, ensuring accuracy.
- Understood how stresses change with position and orientation, which is important for predicting where a beam might fail.

VII. CONCLUSION

This project created a tool to visualize stresses in a cantilever beam with circular cross section. Changing position and stress angle shows how normal and shear stresses vary. Mohr's Circle highlights maximum stresses and critical angles, helping engineers check designs and prevent failure.

REFERENCES

- 1) S. H. Crandall, N. C. Dahl, et al., *An Introduction to Mechanics of Solids*.
- 2) <https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html>.